

TRPO 实验报告

LAMDA-5

June 17, 2026

Abstract

本报告实现并分析了 Trust Region Policy Optimization (TRPO) 算法 [1]。我们完成了 `compute_advantages_from_traj`、`compute_advantages_from_rollout` 和 `_update_policy` 三部分代码补全，并在 Humanoid-v5、HalfCheetah-v5 和 Walker2d-v5 三个 MuJoCo 连续控制环境中进行了实验。实验对比了 rollout 与 trajectory 两种采样方式，以及 advantage normalization 对训练稳定性的影响。结果表明：rollout 在样本利用率 (100%) 上优于 trajectory (62%–71%)，在 HalfCheetah 上最终回报明显更高；trajectory 在 Humanoid 等高维任务中能通过保留完整 episode 边界获得更高质量的 advantage 估计。Advantage normalization 对训练能否成功有决定性影响——在 Walker2d 中，关闭归一化后 surrogate objective 出现量级达 10^{33} 的非有限值，导致策略学习几乎失败。

1 实验设置

本实验在 Humanoid-v5、HalfCheetah-v5 和 Walker2d-v5 三个 MuJoCo Gymnasium v5 环境上比较两种采样方式 (rollout / trajectory) 与是否做 advantage normalization 的影响。主要超参数见表 1。

Table 1: 实验超参数设置

超参数	值
并行环境数	8
每 epoch 交互步数 (rollout)	2000 (共 $8 \times 2000 = 16000$ 个 transition)
轨迹条数 (trajectory)	10
最大 episode 长度	1000
总训练 epoch 数	5000
折扣因子 γ	0.99
GAE 参数 λ	0.95
KL 约束阈值 δ	0.01
共轭梯度迭代次数	10
Critic 优化器	Adam, 学习率 10^{-3}
Critic 更新轮数 / 批大小	5 / 256
策略/价值网络结构	2 层 MLP (64, 64), ReLU 激活; 动作经 Tanh 压缩至动作空间

表 2 汇总了各配置下的关键指标。最终测试回报取训练末期均值; trajectory 模式总交互步数与 rollout 不同, 所报数值为相近交互步数处的测试结果, 以 † 标注。

Table 2: 各配置实验结果汇总

环境	采样模式	Advan Norm	最终测试回报	样本利用率	平均有效步数/epoch
Humanoid-v5	rollout	✓	1083	100%	16000
	rollout		1085	100%	16000
	trajectory	✓	$\sim 655^\dagger$	69.0%	~ 885
HalfCheetah-v5	rollout	✓	4276	100%	16000
	rollout		4608	100%	16000
	trajectory	✓	$\sim 1727^\dagger$	62.5%	~ 10000
Walker2d-v5	rollout	✓	4718	100%	16000
	rollout		355	100%	16000
	trajectory	✓	$\sim 4713^\dagger$	70.8%	~ 6225

2 代码补全

2.1 collect_trajectories 实现

2.1.1 与普通 rollout 的停止条件

普通 rollout 按固定步数截断：每个 epoch 每个并行环境交互满 `interact_per_epoch` 步就返回，不等 episode 结束，buffer 里通常跨多个 episode。`collect_trajectories` 改用轨迹条数截断：外层循环直到 `TrajectoryRollout` 存满 `trajnum` 条轨迹才停止，每条轨迹以环境 `done` 或达到 `max_episode_length` 结束。

2.1.2 每条轨迹保存的信息、终止处理与 padding

每条轨迹保存 `states`、`actions`、`rewards`、`dones`、`log_probs`。`states` 让 actor 重算 $\log \pi_{\theta}(a|s)$ ，也让 critic 估计 V ；`actions` 与 `states` 配合算重要性采样比率；`rewards` 用于反向递推 GAE；`dones` 在递推时乘以 $(1 - d_t)$ 切断 episode 边界的 bootstrap；`log_probs` 是旧策略概率，用来构造 $r_t(\theta) = \exp(\log \pi_{\theta} - \log \pi_{\theta_{\text{old}}})$ 。

环境返回 `done` 或 `truncated` 时调用 `add_traj()` 保存当前轨迹、重置该 env 的缓冲区；未填满 `max_episode_length` 的位置保持零初始化，对应 `masks` 置 0，下游用 `masks==1` 过滤掉 padding。

2.1.3 masks 的作用

`TrajectoryRollout` 预分配形状是 $(\text{trajnum}, \text{max_episode_length})$ ，短轨迹后必有零填充，`add_traj()` 中用 `self.masks[pos,:traj_len]=1` 记录真实长度。如果不 mask 直接计算，padding 的零奖励会污染 GAE，advantage 的均值方差被拉低、归一化随之偏移，样本利用率统计也会虚高，value 拟合的还是无意义的目标。

2.1.4 为何要在每批轨迹收集完后重新初始化环境

不 `reset` 的话，某些 env 会停在 episode 中途，下一轮 trajectory 从非起点状态开始，违背“从初始分布 ρ_0 出发”的假设。这一步不影响 TRPO 的 on-policy 性质：数据始终来自当前策略，更新一次后即清空缓冲区。

2.2 _update_policy 实现

TRPO 的核心理论依赖如下定理：

Theorem 1 (Schulman et al. 2015)

令 $\alpha = D_{\text{TV}}^{\max}(\pi_{\text{old}}, \pi_{\text{new}})$, $\epsilon = \max_{s,a} |A_{\pi_{\text{old}}}(s, a)|$, 则下界成立:

$$\eta(\pi_{\text{new}}) \geq L_{\pi_{\text{old}}}(\pi_{\text{new}}) - \frac{4\epsilon\gamma}{(1-\gamma)^2} \alpha^2.$$

2.2.1 定理各项含义与单调改进条件

$L_{\pi_{\text{old}}}(\pi_{\text{new}})$ 是固定旧策略状态分布下的代理目标 (重要性采样加权 advantage), $\frac{4\epsilon\gamma}{(1-\gamma)^2} \alpha^2$ 是新旧策略 TV 距离引起的惩罚项, 其中 $\epsilon = \max_{s,a} |A_{\pi}(s, a)|$ 是 advantage 的绝对值上界, $\alpha = D_{\text{TV}}^{\max}(\pi_{\text{old}}, \pi_{\text{new}})$ 是新旧策略最大 TV 距离。

只要代理目标提升量超过惩罚项, 即 $L_{\pi_{\text{old}}}(\pi_{\text{new}}) - \frac{4\epsilon\gamma}{(1-\gamma)^2} \alpha^2 \geq \eta(\pi_{\text{old}})$, 下界上升, 策略单调改进。关键的几何直觉: 收益项 $L - \eta(\pi_{\text{old}})$ 对策略偏离量 α 是一次的, 惩罚项 $C\alpha^2$ 是二次的; α 接近 0 时二次项小一个数量级, 所以一定存在小的 α 让收益压过惩罚——”步子迈得够小就一定能改进”在数学上是有保证的。TRPO 顺着这个直觉走: 最大化 L 同时把策略变化压在”信赖域”内:

$$\max_{\pi_{\text{new}}} L_{\pi_{\text{old}}}(\pi_{\text{new}}) \quad \text{s.t.} \quad D_{\text{KL}}^{\max}(\pi_{\text{old}}, \pi_{\text{new}}) \leq \delta.$$

这条定理决定的是 TRPO 的设计坐标系, 而不是它实际能跑多好——深度 RL 里梯度是有限样本估计、策略非线性、advantage 方差大, ”步长足够小”在实践中既要够小又得让训练不至于学不动。

2.2.2 与 Kakade–Langford Theorem 4.1 的处理差异

Kakade–Langford 早期结果只对混合策略 $\pi_{\text{new}} = (1 - \alpha)\pi_{\text{old}} + \alpha\pi'$ 成立, α 是显式的混合系数, 策略类被限制为旧策略与某个候选策略的线性组合。TRPO 把 α 直接定义为 $D_{\text{TV}}^{\max}(\pi_{\text{old}}, \pi_{\text{new}})$, 不再依赖显式混合形式, 因此可以覆盖任意随机策略——包括神经网络参数化的高斯策略, 这是 TRPO 能直接落到深度 RL 上的前提。

2.2.3 Surrogate Objective 的实际形式

代理目标的理论形式 $L_{\pi_{\text{old}}}(\pi_{\text{new}}) = \eta(\pi_{\text{old}}) + \sum_s d^{\pi_{\text{old}}}(s) \sum_a \pi_{\text{new}}(a|s) A_{\pi_{\text{old}}}(s, a)$ 依赖真实的旧策略状态分布 $d^{\pi_{\text{old}}}$, 无法精确得到。工程上用旧策略采样数据做 Monte Carlo 估计, 并通过重要性采样修正:

$$\hat{L}(\theta) = \frac{1}{N} \sum_{t=1}^N \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t,$$

$\hat{A}_t$ 由 GAE 估计。代码中先保存采样时的 `old_log_probs`, 更新时计算 `ratio = exp(new_log_probs - old_log_probs)`, 最后 `surrogate = (ratio * advantages).mean()`。

在 $\theta = \theta_{\text{old}}$ 处 $\hat{L}$ 与 η 的梯度相等（两者相差一个常数 $\eta(\pi_{\text{old}})$ ），所以沿 $\hat{L}$ 的梯度走极小步就等价于沿 η 走。

2.2.4 为什么约束 KL 而非 TV，为什么需要线搜索

TRPO 实际约束的是 KL 而非定理中的 TV。由 Pinsker 不等式 $D_{\text{TV}}(p, q) \leq \sqrt{\frac{1}{2}D_{\text{KL}}(p||q)}$ ，约束 KL 就能隐式控制 TV，且约束更紧；KL 对高斯策略有闭式且处处可微，TV 在连续空间不光滑、难以做梯度和二阶近似。在当前参数邻域，对 KL 做二阶泰勒展开：

$$D_{\text{KL}}(\theta_{\text{old}}, \theta) \approx \frac{1}{2}\Delta\theta^\top F \Delta\theta, \quad F = \nabla_\theta^2 D_{\text{KL}}(\theta_{\text{old}}, \theta) \Big|_{\theta=\theta_{\text{old}}} = \mathbb{E} \left[\nabla_\theta \log \pi_\theta \nabla_\theta \log \pi_\theta^\top \right] \Big|_{\theta_{\text{old}}}$$

其中 F 是 Fisher 信息矩阵，信赖域约束因此化为椭球约束 $\frac{1}{2}\Delta\theta^\top F \Delta\theta \leq \delta$ 。线性化 surrogate 后的最优方向就是自然梯度 $\Delta\theta_{\text{base}} = F^{-1}g$ ——这其实就是自然策略梯度 (NPG) 的思路 [4, 3]：不在参数欧氏距离下走普通梯度，而是在策略分布的 KL 几何下修正方向；后续 MDPO 等工作也把 TRPO、PPO 放在 mirror descent 框架下统一比较 [5]。满足约束的最大步长为 $\Delta\theta^* = \sqrt{2\delta/(\Delta\theta_{\text{base}}^\top F \Delta\theta_{\text{base}})} \cdot \Delta\theta_{\text{base}}$ 。

神经网络参数太多，显式构造并求逆 F 不现实。代码中只实现 Hessian-vector product: `kl_product(v)` 计算 Fv ，再用共轭梯度近似解 $Fx = g$ ，得到 $x \approx F^{-1}g$ ；实现里 `return product + vector * 0.1` 相当于给 F 加阻尼 λI 改善条件数。最后还要做回溯线搜索：沿 $\theta_k = \theta_{\text{old}} + \beta^k \Delta\theta^*$ 逐步缩小步长，直到实际 $\text{KL} \leq \delta$ 且 surrogate 不下降；两个条件缺一不可，全部候选失败则回滚参数。

3 探索部分

3.1 不同环境的训练难度

三个环境的难度差异很明显：rollout 末期平均回报分别约为 Humanoid 1083、HalfCheetah 4276、Walker2d 4718。HalfCheetah 和 Walker2d 的曲线持续上升并收敛到较高水平；Humanoid 因为 348 维状态和复杂的身体协调要求，提升慢、最终值也低，同样超参数下很难追上。

3.2 Trajectory 与 Rollout 模式下 advantage 与 return 的计算差异

两种模式都使用 GAE: $\delta_t = r_t + \gamma(1 - d_t)V(s_{t+1}) - V(s_t)$, $A_t = \delta_t + \gamma\lambda(1 - d_t)A_{t+1}$, $R_t = A_t + V(s_t)$ 。公式一致，差异在于数据组织与边界处理：

维度	trajectory	rollout
数据形状	(trajnum, max_ep_len, ·) 有 padding	(num_envs, pos, ·) 无 padding
序列单位	一条完整 episode	一个 env 的固定步数片段，跨多 episode
有效长度	每条 valid_len = masks[i].sum()	所有 env 均为 pos
末尾 bootstrap	next_value=0 (自然/截断都按结束处理)	next_value=0 (截断也当结束，理论有偏)
中间 done	仅最后一步可能 $d = 1$	中间可多次 $d = 1$ ，靠 $(1-d)$ 切断
过滤	reshape(-1)[mask] 去 padding	直接 reshape(-1)，全部有效
返回长度	$\sum_i \text{valid_len}_i$ ，每轮可变	num_envs $\times$ pos，固定

trajectory 对每条完整轨迹反向递推、用 masks 去 padding，episode 边界更精确；rollout 对固定长度片段反向递推，所有 transition 都参与更新，但若片段末端不是自然终止，把末端 value 当作 0 就会引入 truncation bias。本项目两种模式都把最后一步 next value 置零，这对自然终止的 trajectory 合理，对截断的 rollout 不完全合理——这一差异部分解释了为什么 trajectory 在高维任务中有时表现更好。严格实现 rollout 应额外保存 last_value = $V(s_{\text{pos}})$ 做 bootstrap，interact_per_epoch 越小这一偏差越显著。

3.3 Advantage Normalization 的影响

3.3.1 两组配置的回报曲线差异

固定 rollout 模式、只切换 advan_norm，三个环境差异很大 (图 1)：HalfCheetah 不归一化反而更高 (4608 vs 4276)，Humanoid 两组几乎一样 (1085 vs 1083)，Walker2d 完全相反——归一化组达到 4718，不归一化长期停留在 355，基本没学到。

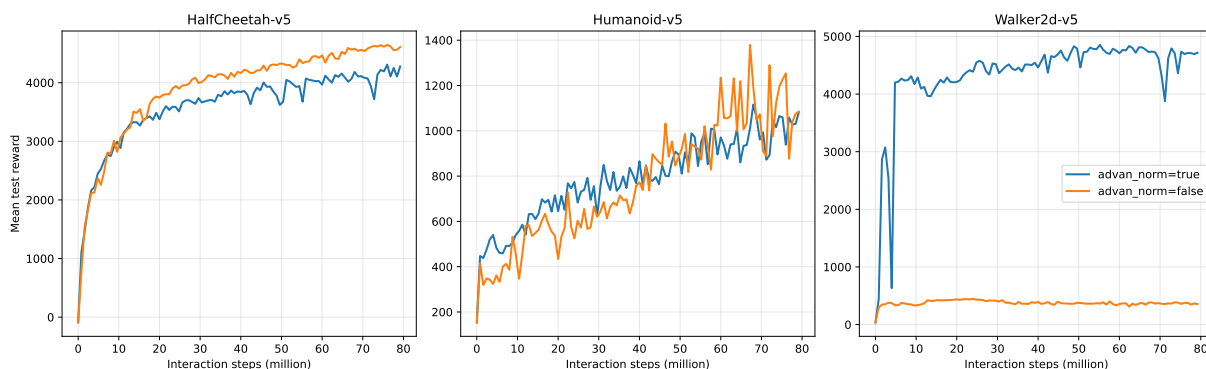


Figure 1: 三个环境中 advantage normalization 对测试平均回报曲线的影响

3.3.2 训练是否更稳定

Walker2d 不归一化时 surrogate 只有 86.6% 的记录是有限值，最大绝对值 9.83×10^{33} ；平均 KL 仅 0.00213，约 4% 的更新中 KL 接近 0，说明大量更新退化或失败。归一化后

surrogate 全部有限，平均 KL 升到 0.00881，真正用满了 $\delta = 0.01$ 的信赖域。Humanoid 中，归一化组最后十次回报标准差为 57.1，不归一化组为 123.8（图 2）。

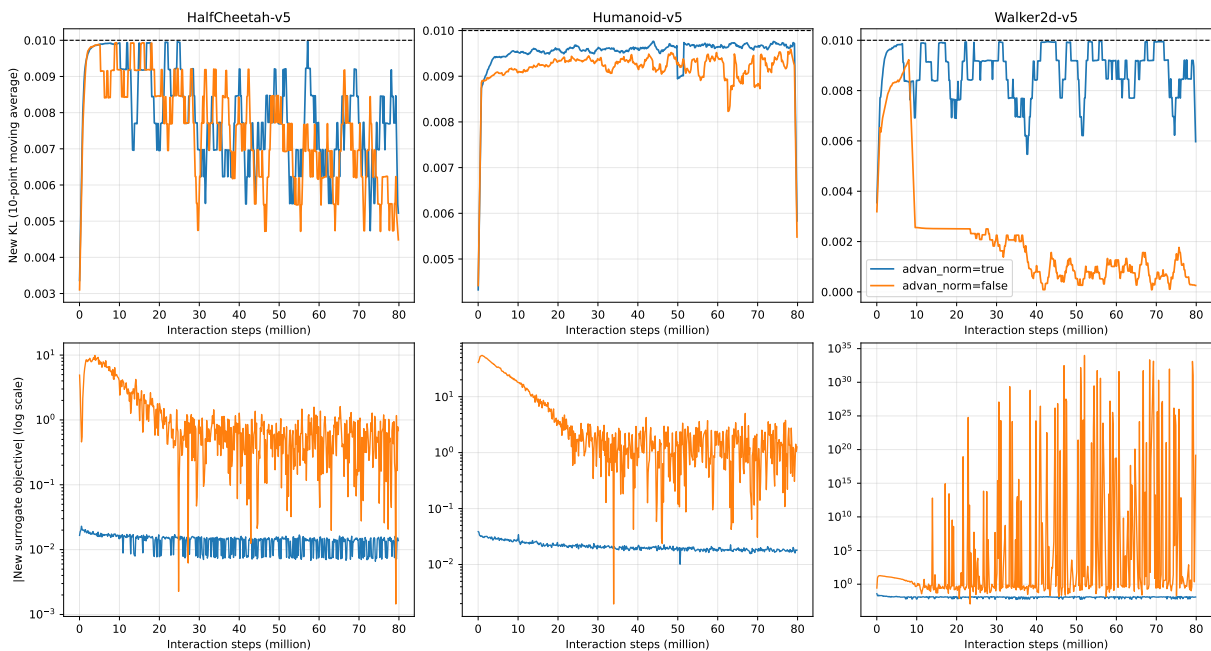


Figure 2: Advantage normalization 对 KL divergence 与 surrogate objective 的影响

3.3.3 是否总是带来性能提升

不是。HalfCheetah 不归一化反而略高一点。归一化更像数值鲁棒性的兜底——TRPO 已经用 KL 约束限制了更新幅度，advantage 尺度本来稳定时再减去 batch 均值反而改变了有限样本下的梯度方向、引入统计噪声；但一旦 advantage 量级失控（Walker2d），归一化就是必须。

3.4 Trajectory 与 Rollout 的样本利用率

3.4.1 两种模式实际使用的 transition 数

rollout 配置 8 个并行环境、`interact_per_epoch=2000`，每次更新固定使用 $8 \times 2000 = 16000$ 个 transition；trajectory 每次收集 10 条轨迹，有效步数由 `masks.sum()` 决定。5000 次更新平均下来，HalfCheetah、Humanoid、Walker2d 实际使用 transition 约 10000、885、6225（图 3）。

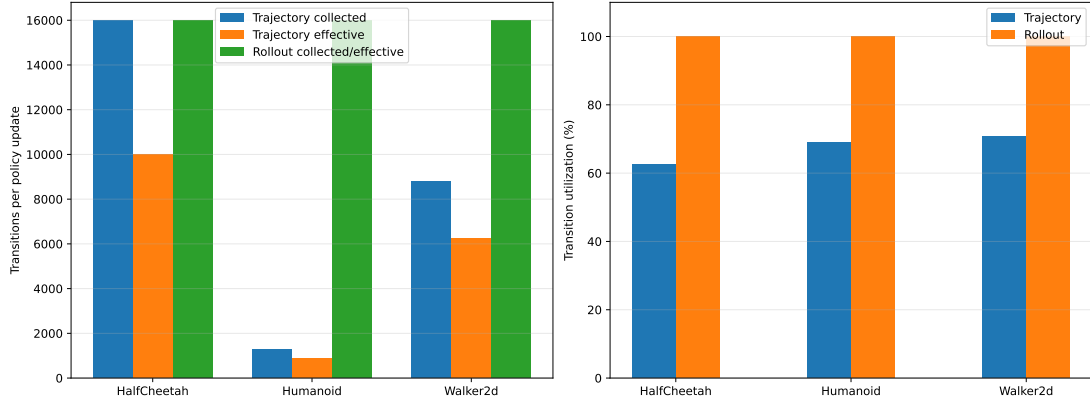


Figure 3: Trajectory 与 rollout 每次更新的 transition 数量与样本利用率

3.4.2 哪种模式利用率更高，差异最明显在哪个环境

rollout 利用率始终 100%，trajectory 三个环境分别只有 62.5%/69.0%/70.8%。按百分比看 HalfCheetah 差距最大：它没有提前终止机制，episode 通常都跑满 1000 步上限、并行环境结束时间高度同步，存满 10 条后剩余几条整批被丢弃。但按绝对有效步数看，最严重的是 **Humanoid**——只有 ~ 885 步，与 rollout 的 16000 相差一个数量级。原因是 Humanoid 的 `healthy_z_range` 约束严格，训练早期智能体极易摔倒，单条 episode 通常只能跑几十步即被 `terminated`，而 TrajectoryRollout 仍按 `max_episode_length=1000` 预分配 buffer，绝大部分位置都是 padding。Walker2d 介于二者之间。两种视角合起来说明：trajectory 的“样本利用率劣势”既会被并行环境长度同步性放大（HalfCheetah），也会被环境早停概率放大（Humanoid），后者对训练效率的影响更致命。

3.4.3 利用率高是否意味着效果更好

不一定。相同交互步数处，HalfCheetah rollout (4276) 远好于 trajectory (1727)；Humanoid 在 6.34×10^6 步处 trajectory (655) 反而高于 rollout (459)；Walker2d 在 4.33×10^7 步处两者接近 (4713 vs 4657)。rollout 不丢样本但有截断偏差，trajectory 浪费样本但 episode 边界干净，是数量与质量的权衡。

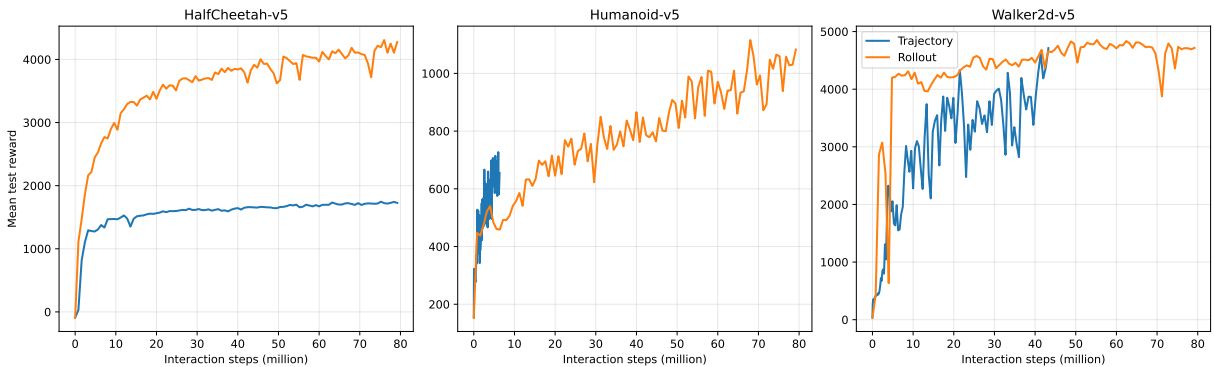


Figure 4: Trajectory 与 rollout 的测试平均回报曲线

3.5 训练过程诊断：critic loss、step size 与 backtrack

为了把 §3.3 的稳定性结论落到 TRPO 更新链路的每一环上，从日志取出 value/loss、actor/stepsizes、actor/backtrack_step 画在图 5，关键统计见表 3。

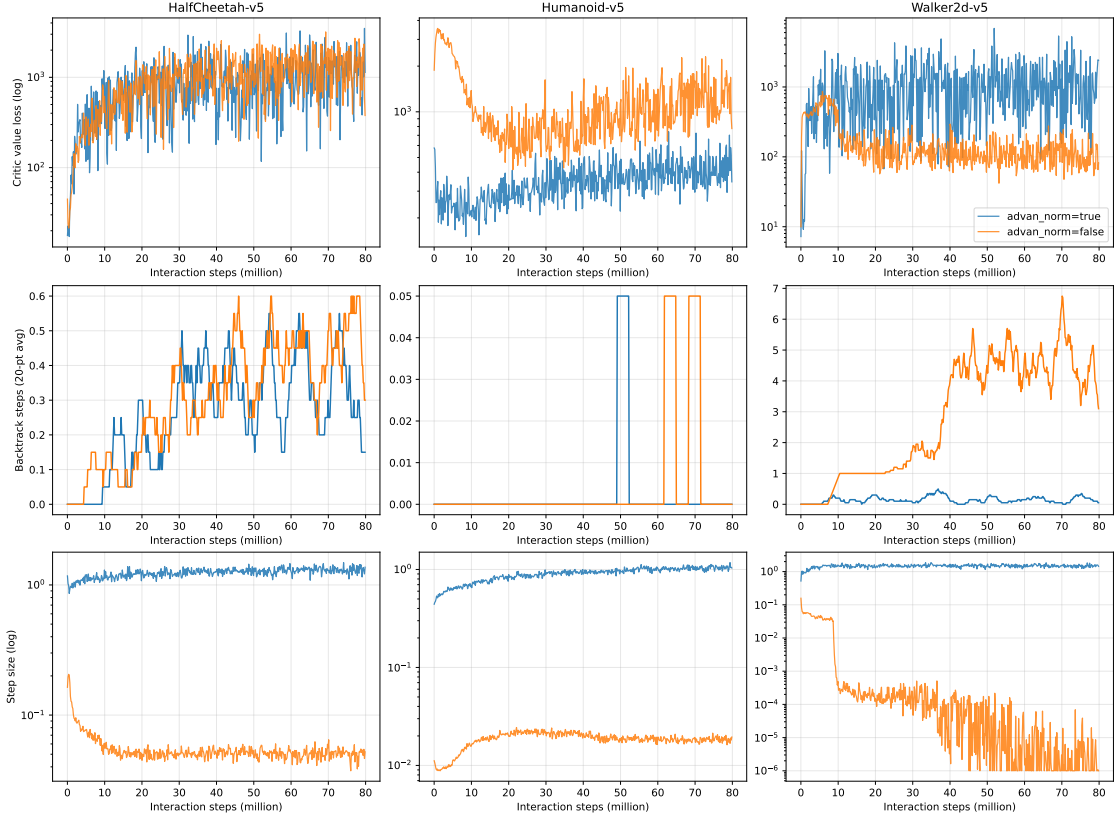


Figure 5: 三个环境、advan_norm true/false 下的 critic value loss (log 轴)、回溯步数 (20-pt 滑动平均) 与自然梯度步长 (log 轴)。

Table 3: 诊断指标统计 (rollout 配置)

环境	advan_norm	value loss 均值	回溯步均值	回溯步最大	step size 均值
HalfCheetah	true	903.9	0.26	1	1.24
HalfCheetah	false	1018.6	0.30	1	5.54×10^{-2}
Humanoid	true	349.1	0.00	1	0.90
Humanoid	false	1130.2	0.00	1	1.85×10^{-2}
Walker2d	true	1093.8	0.14	1	1.47
Walker2d	false	170.3	2.86	9	5.34×10^{-3}

三个归一化配置的回溯均值都在 $0 \sim 0.3$ ，最大也只得 1，说明二阶近似下 CG 解出的方向基本第一次就被采纳。关掉归一化后 step size 被压低 $1 \sim 2$ 个数量级 (HalfCheetah $1.24 \rightarrow 0.055$ 、Humanoid $0.90 \rightarrow 0.019$ 、Walker2d $1.47 \rightarrow 0.0053$)，因为 advantage 未中心化时 $\Delta\theta_{\text{base}}^\top F \Delta\theta_{\text{base}}$ 被放大，按 $\sqrt{2\delta/\cdot}$ 反推的步长自然被压缩——与

§3.3 的 Walker2d 平均 KL 0.00213 完全自愈。Walker2d 不归一化是唯一回溯频繁失败的配置（均值 2.86、最高 9 次即全部回退），对应 surrogate 炸到 10^{33} ；Humanoid 不归一化时 critic loss 从 349 升到 1130，actor 却因 step size 太小没法把信号转化为策略改进；HalfCheetah 因为 advantage 尺度本来就稳定，归一化关掉只是 step size 略降，回报反而更好。

为何这一结论不依赖多 seed. Walker2d 不归一化时 surrogate 13.4% 非有限、最大 9.83×10^{33} ，回溯均值 2.86（最大触顶 9 即参数回滚），KL 均值 0.00213 不到 δ 的四分之一——这是系统性数值崩溃，与 run-to-run 随机性不在同一量级，单 seed 即可定性。HalfCheetah / Humanoid 上归一化与否的差异在波动范围内，故本报告措辞保守（“略降”、“持平”），不做过强声明。

3.6 扩展环境：Ant-v5 与 Hopper-v5

为验证主结论的普适性，我们补充了 Ant-v5 和 Hopper-v5（图 6）。Ant-v5 上 rollout + advan_norm=false 末期回报约 5433（best 5562）、trajectory + advan_norm=true 约 4891（best 5126），均明显超过 Humanoid；关闭归一化未出现 Walker2d 式崩溃，进一步支持 §3.3 的判断 归一化的必要性依赖环境的 *advantage* 量级，而非默认开启。Hopper-v5 上 rollout + norm、trajectory + norm 的 best 接近（3712 vs 3525–3633），但末期回报均回落（1791、1881–1988），呈”高峰后崩溃”模式——与 Hopper 摔倒即终止的稀疏失败结构有关，超出本文消融变量的影响范围。

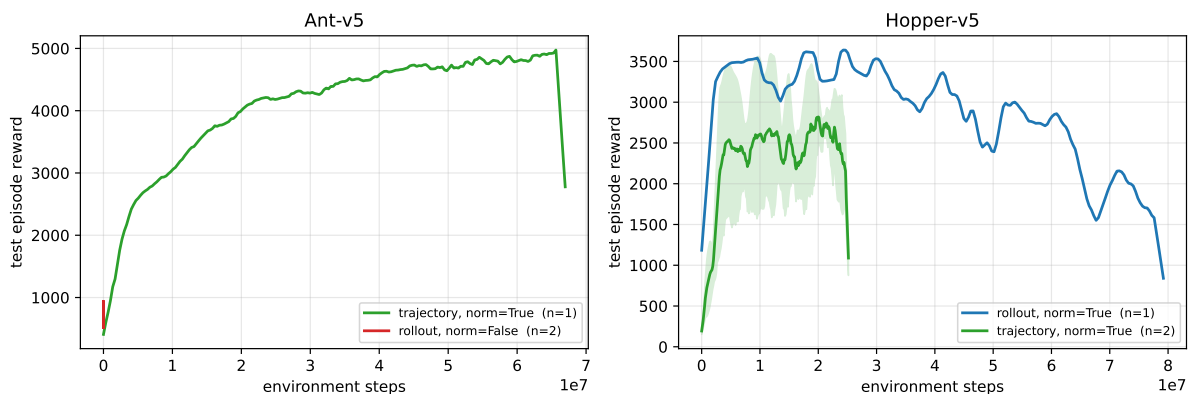


Figure 6: Ant-v5 与 Hopper-v5 上不同采样模式 / 归一化配置的测试回报曲线（多 run 配置显示均值与 min-max 阴影）。

扩展后，§3.1（环境难度差异显著）、§3.2–§3.4（rollout 与 trajectory 各有优势，利用率不等于回报高）、§3.3–§3.4（normalization 的必要性依赖环境 *advantage* 量级、关掉后 step size 与 backtrack 行为会同步退化）这三条主结论在 5 个 MuJoCo 环境上保持一致。

4 反思

4.1 collect_trajectories 是否改变 TRPO 的 on-policy 性质

不改变。两种模式都用当前策略采数据、更新一次后清空缓冲区，区别只是停止条件和数据组织形式：rollout 按固定步数截断，trajectory 按完整轨迹条数截断。数据始终来自最新策略，没有跨策略复用。

4.2 只保留一种采样方式：rollout 还是 trajectory

我会选 rollout。它每次固定用满 16000 个 transition，批量稳定、实现简单，HalfCheetah 上明显更好，Walker2d 上最终性能也很好。trajectory 在 Humanoid 等高维任务相同步数下偶尔会高一些，但综合下来 rollout 更稳定。如果继续做，给 rollout 末端加上正确的 value bootstrap 能进一步缩小截断偏差。

4.3 完成报告后对 TRPO 算法的新认识

从我个人的感受来说，TRPO 能通过数学不等式来保证效果的稳定改进，还是挺 promising 的。整体的证明思路有点像切比雪夫不等式：核心定理里用 $\alpha^2 = D_{TV}^{\max}(\pi_{old}, \pi_{new})^2$ 乘一个系数，把真实回报相对代理目标的偏离框在一个二次的“整体偏差”量里，跟切比雪夫用方差去框尾巴偏离的精神是相通的，都是拿一个整体性的偏差量去保证偏离不会太多。再加上零点附近其实一次比二次快这件事，“步子迈得够小就一定能改进”在数学上就站得住了。有趣的是 TRPO 不只是单纯的理论推进，虽然理论确实挺完善的，但是从采样再到最后用均值替代期望，其实是一种效率，是一种工程和理论的平衡。整个算法读下来还有一种几何直觉，有点像在流形上慢慢上升的感觉，每一步都不走很远，但保证在往上走。

这种感受其实是写完报告之后才慢慢成型的。

最初我只把 TRPO 记成“用 KL 限制策略更新”，单调改进定理是它的理论外壳，但这套推导依赖精确期望、精确 advantage 和理想化的约束求解——一旦进入神经网络和有限样本，理论本身并不能直接回答“为什么 vanilla policy gradient 不行、TRPO 就行”。普通梯度法配上足够小的步长也有局部上升的直觉，问题在于这个步长往往小到训练效率很差，而高方差采样让“足够小”很难判断。我现在更愿意把 TRPO 看成 natural policy gradient / mirror descent 的工程化版本：用 KL 和 Fisher 矩阵重新度量“一步走多远”，再用 Hessian-vector product、共轭梯度、line search 让这件事在大网络上能算出来。

这次实验也让我对 implementation choice 的影响有了更直观的感受。Walker2d 关掉 advantage normalization 之后 surrogate 直接出现非有限值，一个看似只是标准化的细节就能决定能不能训。同样地，trajectory 的 masks、rollout 末端是否 bootstrap、buffer 满时 pos 的归零都不写在论文主公式里，但都会直接影响 advantage、KL 和梯度的可信度，这一点和 [6] 的观察一致：深度 RL 里代码级细节会显著左右算法表现。

4.4 实现改进建议、README 卡点与难度感受

实现层面的改进建议. 最直接的一条是 rollout 末端的 bootstrap 缺失。当前实现把每个 rollout 片段最后一步的 next value 直接置零，但这一步通常并不是真正的 episode 结束，应该用 $V(s_{\text{pos}})$ 做 bootstrap。改法很轻：在 ReplayBuffer 里多存一个 last_value 给 $t==\text{pos}-1$ 处用即可。interact_per_epoch 越小，这一截断偏差越显著，§3.2 的对比也佐证了这一点。

卡点与 workflow 反思. 就理论本身而言，TRPO 的推导初读需要花点时间消化，但只要抓住“代理目标加 TV 惩罚、再换成 KL”这条主线，每一步推到下一步都还算自然，题目本身的设置也比较循序渐进。真正卡时间的几乎都是工程和流程上的细节：trajectory 模式下 dones 把“截断”和“自然终止”等同对待，一开始以为代码能区分 terminated / truncated，排查 GAE 时绕了弯；kl_product / get_flat_grad 里二阶计算图的 retain_graph / create_graph 组合在 README 里完全没提，初次跑 CG 直接报“trying to backward through graph a second time”，建议补一段二阶梯度的注意事项。

另一个事后看很明显的卡点其实在 workflow：这次跑实验是本地一组组排队、间歇看结果再调下一组，拉长了整体的 wall-clock 时间。下次类似规模的消融，应该一开始就把所有配置打包成脚本一次性挂到服务器上并行跑，等结果统一回收做分析，而不是边跑边等。

难度感受. 整体偏中等。GAE 计算与 buffer 处理是体力活，看清楚 shape 就不难；二阶部分（Fisher 向量积、共轭梯度、线搜索）跟着 README 和已有代码框架走下来也并不算难。实验报告部分的难度主要来自实验脚本的多组合（2 种采样 $\times$ 2 种归一化 $\times$ 5 个环境）以及日志解析，逻辑上不复杂，但跑全要花时间。综合而言，理论推导和工程实现难度都适中，实验分析部分难度较低。

5 总体结论

- (1) 三个主要环境难度差异明显：HalfCheetah 最易收敛 (4276), Walker2d 最高 (4718), Humanoid 因高维状态最难 (1083); 补充的 Ant-v5 (~ 5400) 与 Hopper-v5 (~ 3700 峰值后回落) 进一步把样本说明扩展到 5 个 MuJoCo 环境。
- (2) rollout 样本利用率 (100%) 高于 trajectory (62% $\sim$ 71%)，但 Humanoid 和 Walker2d 在相同交互步数下 trajectory 反而略好——利用率高不等于训练效果好。
- (3) advantage normalization 是数值稳定性的兜底：Walker2d 关掉后 surrogate 出现 10^{33} 量级的非有限值、step size 被压缩到 5.34×10^{-3} 、回溯均值升到 2.86 (最高 9 次即回滚); 但 HalfCheetah 不归一化反而更高，Ant-v5 也没崩，说明归一化的必要性依赖环境 advantage 量级而非应当默认开启。
- (4) TRPO 的理论给出单调改进的设计原则，但实际表现还取决于 advantage normalization、rollout 末端 bootstrap、CG 阻尼等实现细节。可以把 TRPO 视为 NPG

/ mirror descent 的工程化版本，理论和实现两侧需要一起看；§3.4 的诊断（value loss / step size / backtrack）也印证这条更新链路是相互耦合的，不能只看主公式。

References

- [1] Schulman, J., Levine, S., Abbeel, P., Jordan, M., & Moritz, P. (2015). Trust Region Policy Optimization. In *Proceedings of the 32nd International Conference on Machine Learning* (ICML 2015), pp. 1889–1897. PMLR.
- [2] Kakade, S., & Langford, J. (2002). Approximately Optimal Approximate Reinforcement Learning. In *Proceedings of the 19th International Conference on Machine Learning* (ICML 2002), pp. 267–274.
- [3] OpenAI. Spinning Up: TRPO. <https://spinningup.openai.com/en/latest/algorithms/trpo.html>
- [4] Kakade, S. M. (2001). A Natural Policy Gradient. In *Advances in Neural Information Processing Systems 14*.
- [5] Tomar, M., Shani, L., Efroni, Y., & Ghavamzadeh, M. (2020). Mirror Descent Policy Optimization. *arXiv preprint arXiv:2005.09814*.
- [6] Engstrom, L., Ilyas, A., Santurkar, S., Tsipras, D., Janoos, F., Rudolph, L., & Madry, A. (2020). Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO. *arXiv preprint arXiv:2005.12729*.